CNENGR115 Computers and Control

Computers

Lecture 2/3 – Choices, architecture, relevance and I/Os

Dr Zhongming Zhang

School of Engineering

Lancaster University

Embedded Systems

Lancaster University

Revision of Lecture 1:

- A *microcontroller* (μC) is one of a family of embedded systems combining a microprocessor, timers, ADC and other features on *one piece of silicon* to control things from 'inside'.
- μCs enable the power of small, lower-power programmable devices to be used cost-effectively, reliably and unobtrusively.
- These devices can be programmed in a variety of languages. This code is converted into *machine code* and moved into non-volatile flash memory on the device; such memory can be re-programmed virtually endlessly without issue.
- Some programming environments enable the operation of our code to be *simulated* prior to transfer to the device, and also to be *debugged* on the device: simulation is generally fast, debug is slow.

Focus of this lecture: Embedded system design issues

- There are two important design issues for embedded system engineers:
  1. Which device family?
  2. What peripherals are necessary for a given application?

Cost?

Use in existing products?

Familiarity?

Compatibility with the application environment?

Ease of integration?

Peripherals?

Obsolescence?

Once you've chosen your device…more options to consider

Is there a variant dedicated to my application?

Speed?

Package?

Power consumption?

Memory capacity?

I2C?

GPIO?

Serial out?

On-board ADCs: speed, resolution?

First – A digression about architectures…

- Fundamentally, μCs do the following:
  1. *Receive* information (signals).
  2. *Encode* that information as a number and store it in volatile memory (RAM).
  3. *Process* this information (number) according to a program (in ROM), and store the result in memory (RAM).
  4. *Output* information (as an analogue or digital signal) with which to do useful work.
- It's about signal processing, NOT power.

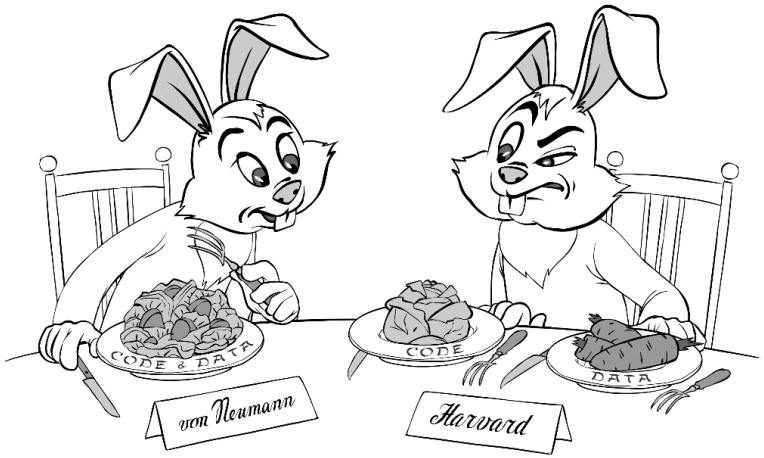
Digression continues…

- How do we process this information?
  - Analogue computers, i.e. comprising a specific sequence of electronic components, can only solve one problem over and over again as specified by the specific sequence.
  - The *Universal Turing Machine* stores a sequence of instructions (program) in the same medium (tape) as the sequence of numbers (data) on which the program is to act.
  - This heralded the possibility of flexibility in both rather than only being able to process different sets of numbers in a single way.

Alan Turing

And continues…

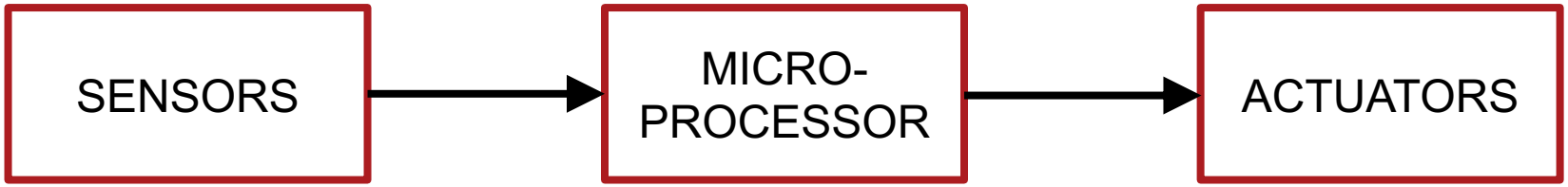
- Turing showed that we can store the instructions and data in the same place (memory).
- Either:
  - *Von Neumann* architecture: comprises a single bus for *data* and *instructions*, enabling efficient memory use but both cannot be retrieved at the same time (therefore slow).
- Or separately:
  - *Harvard* architecture: less memory efficient, read-only instruction memory (robust), both can be retrieved simultaneously.



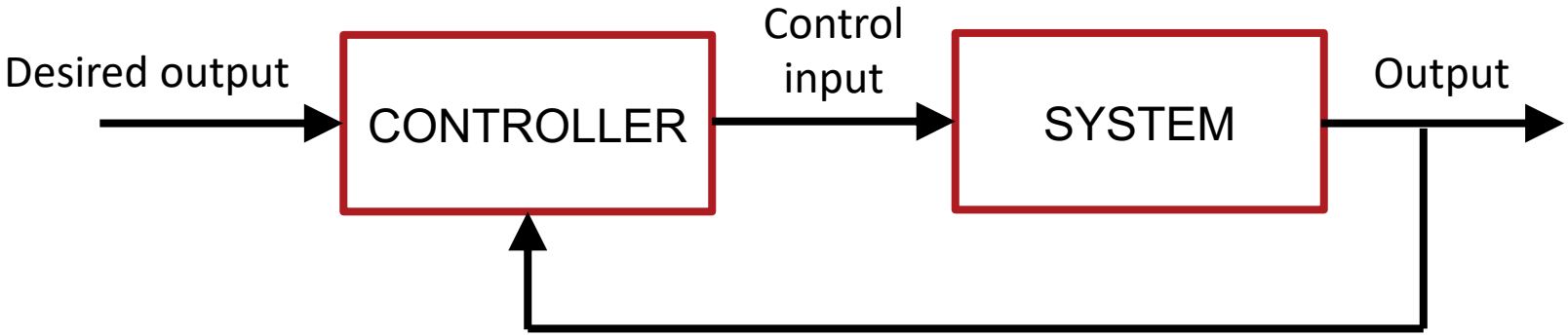
http://ithare.com

John Von Neumann

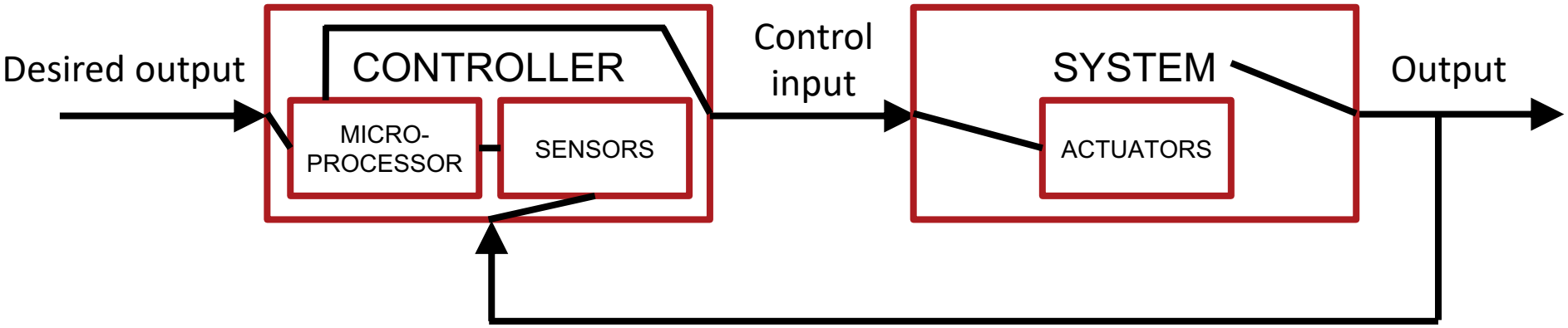
How is this relevant to Control Engineering?



Ref: C520B_Arduino_Projects_by_Chumbaka.pdf



How is this relevant to Control Engineering?

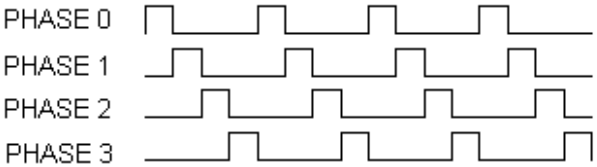


What Actuators do we have access to?

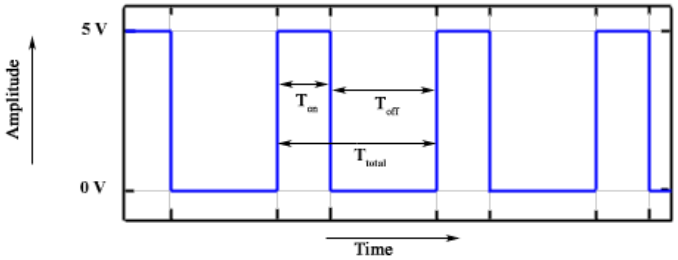
- Tri-colour LED module
- Ultrasound module
- Servo motor
- 7-segment display
- 8x8 matrix display
- LCD module
- OLED display
- Stepper motor
- LED
- Active buzzer
- Passive buzzer

- These all **require** a digital signal
  - Some are simply ON or OFF, single or multiple digital lines.
  - Some can be varied using a pulse train or Pulse Width Modulation (PWM) or frequency modulation or encoded data (communication protocols, e.g. SPI or I2C).

http://www.hteck.ca/electronics/s-motor-tster/sm-tester.html



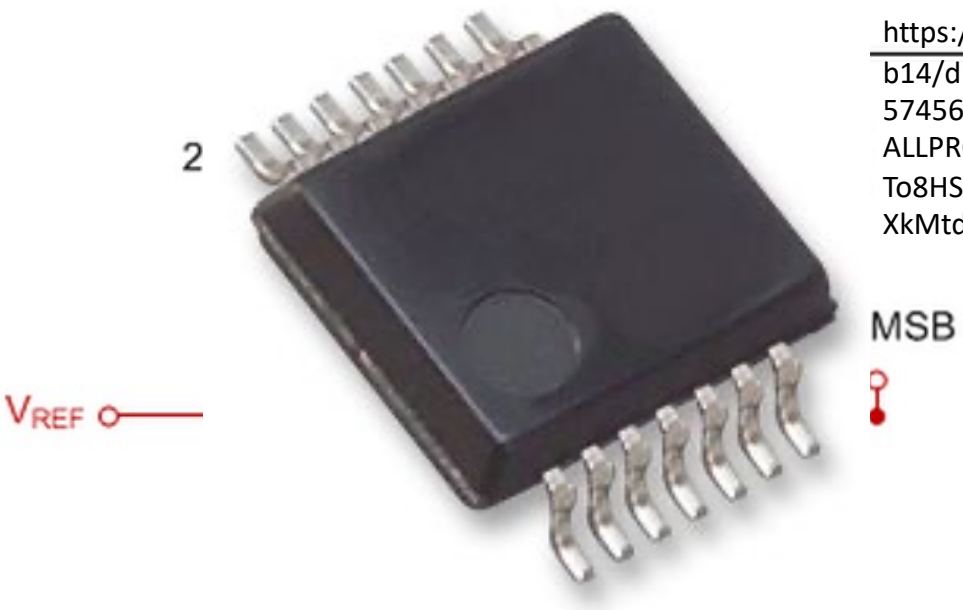
WAVE DRIVE WAVEFORM



https://www.tutorialspoint.com/arduino/arduino_pulse_width_modulation.htm

But what if we use an actuator that **requires** an analogue signal?

- A voltage that varies from 0 V – 5 V
  - We need to use a Digital-to-Analogue Converter (DAC).
  - Op-amp configured as voltage follower, switches that move between V_{REF} and ground.



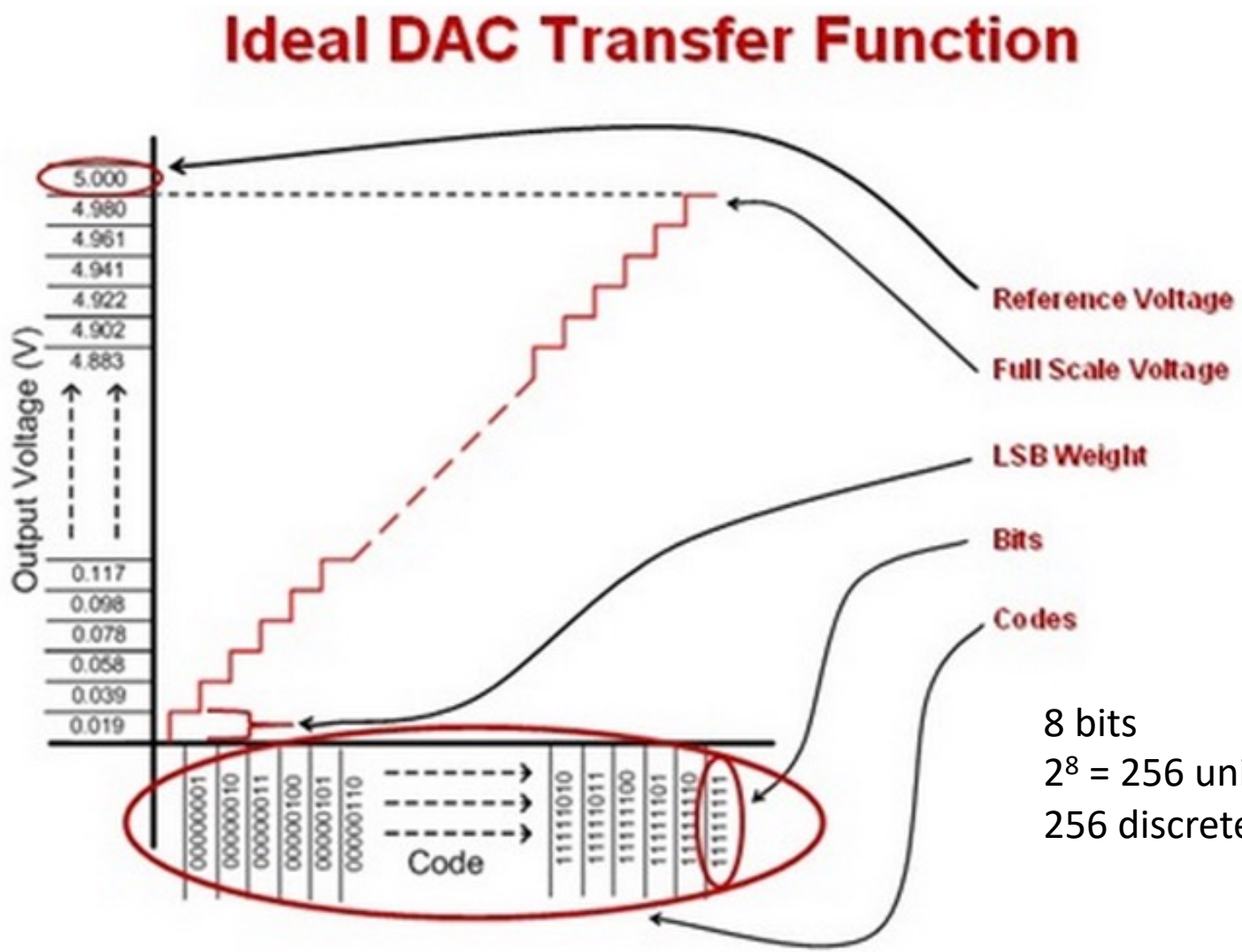
https://uk.farnell.com/rohm/bh2227fv-e2/ic-dac-4ch-type-ssop-b14/dp/1716111?gross_price=true&CATCI=pla-57456867730&CAAGID=14406255429&CMP=KNC-GUK-GEN-SHOPPING-ALLPRODGROUPS&CAGPSPN=pla&gclid=Cj0KCQjw3KzdBRDWARIsAIJ8TMTo8HSgXPt6PQeejGPYexEqh7rOVlcVLj0Mz-RYMcpJJ4dD-XkMtdcaArIqEALw_wcB&CAWELAID=120173390001300038

https://www.cnblogs.com/shangdawei/p/4821663.html

https://learn.sparkfun.com/tutorials/voltage-dividers

$$V_{out} = V_{in}\frac{R_2}{R_1 + R_2}$$

But what if we use an actuator that **requires** an analogue signal?



8 bits

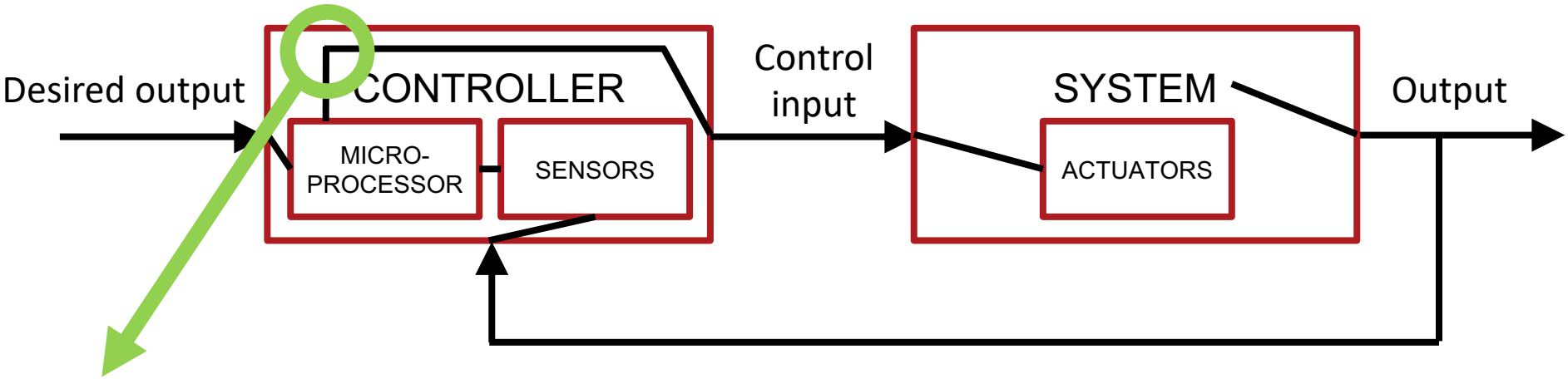
2^8 = 256 unique codes,

256 discrete voltages

But what if we use an actuator that **requires** an analogue signal?

- The Arduino Uno **does not** have a built in DAC!

How is this relevant to Control Engineering?



- Depending on what actuator we use…
- The DAC if the actuator requires an analogue signal would need to be implemented here.
- Otherwise we just use one of the digital modes described earlier (e.g. PWM).

What Sensors do we have access to?

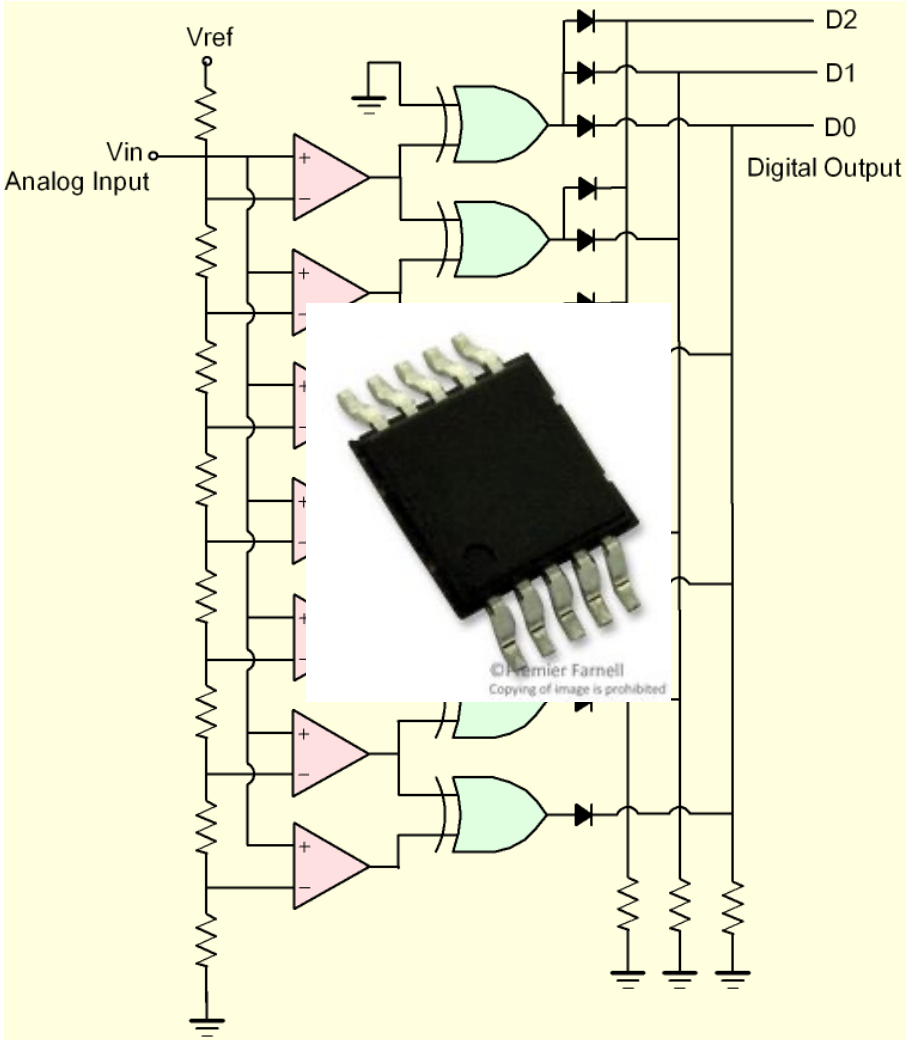
- Thermistor
- Flame sensor
- Temperature sensor
- Photoresistor
- Potentiometer

- These all **provide** an analogue signal
  - A varying resistance that induces a varying voltage for example.

- Touch module
- Ultrasound module
- Vibration (tilt) sensor
- IR receiver
- Key switch

- These all **provide** a digital signal
  - Some are simply ON or OFF.
  - Some produce a pulse width modulated signal or encoded data.
  - Simply use the digital I/Os, the Arduino Uno has 14 (of which 6 provide PWM output).

So if we use a sensor that **provides** an analogue signal?



- A voltage that varies from 0 V – 5 V
  - We need to use a Analogue-to-Digital Converter (ADC).
  - E.g. Voltage divider, op-amps as comparators, XOR gates (priority encoder).

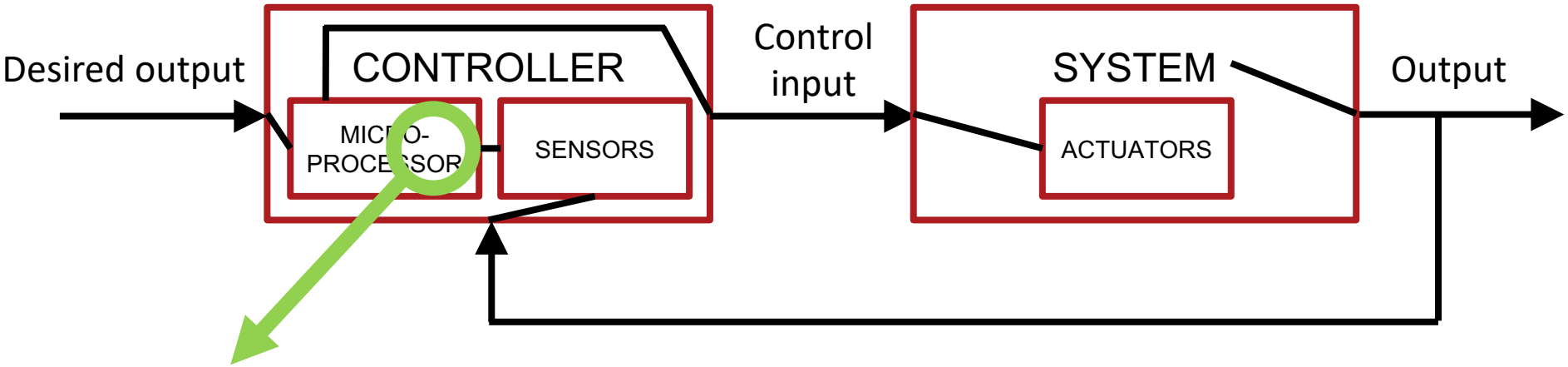
https://uk.farnell.com/texas-instruments/ads1118idgsr/adc-16bit-temp-sensor-10msop/dp/1947871

https://www.hardwaresecrets.com/how-analog-to-digital-converter-adc-works/6/

So if we use a sensor that **provides** an analogue signal?

- The Arduino Uno **has six** built in ADCs!
  - Resolution 10 bit (2^{10} = 1024).
  - Therefore, the **raw values** from the ADC that you observe using the Serial Plotter will range from 0 – 1023.
  - This only represents the varied voltage on the ADC input, you have to convert it into **something meaningful** (e.g. temperature, speed, ventilation rate….).
  - Check out https://store.arduino.cc/arduino-uno-rev3

How is this relevant to Control Engineering?



- Depending on what sensor we use…
- The ADC (in our case) if the sensor provides an analogue signal is built-in to the µC hardware.
- Otherwise we just use one of the digital I/Os.

Summary

- Embedded system design choices/issues
- Fundamental use of µC
- Their role in Control Engineering
- Actuators -> Getting something useful out
  - Data, heat exchange, motion, light
  - To inform the user or **stimulus to the plant/system**
- Sensors <- Getting useful information in
  - Data representing temperature, velocity, luminosity
  - To inform the **controller**

Private study

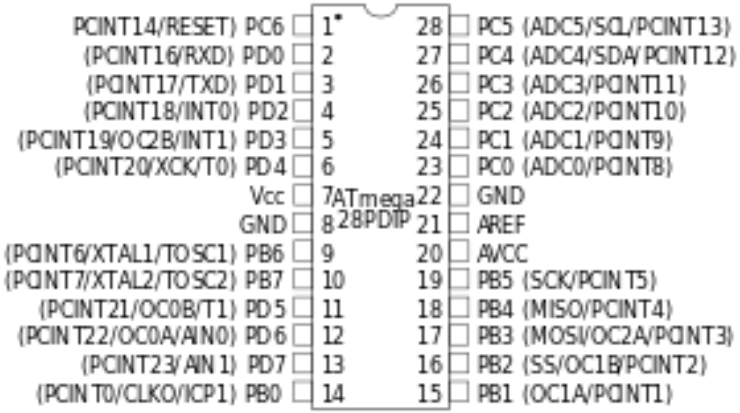
- The following slides are additional slides to continue the digression…

And continues…in terms of *instruction set* architectures (ISAs)

- A *reduced instruction set computer* (RISC):
  - Memory access limited to specific instructions.
  - Separate memory load instructions.
  - Number of cycles per instruction is *reduced*, thus far fewer transistors.
  - Atmel AVR, ARM
- A *complex instruction set computer* (CISC):
  - Single commands can comprise several operations.
  - A range of cycle operations per instruction, no separate memory load instructions.
  - MC6800, 8051, Z80.

Device families

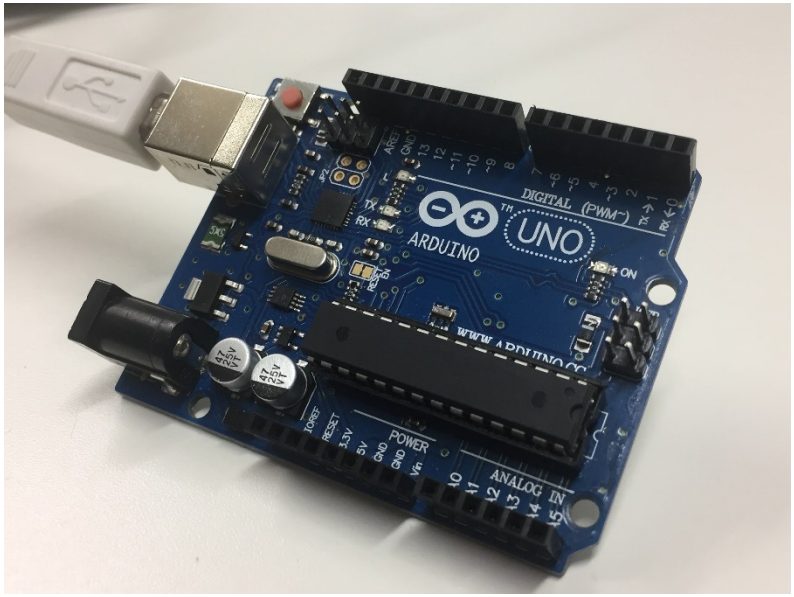
- For example:
  - *Atmel* is the manufacturer
  - *AVR* is the family
  - *ATmega328P* is the specific device or model
  - (*Arduino Uno* is the programming unit)



ATmega328P dual-in-line (DIL) package, Wikipedia

AVR family details

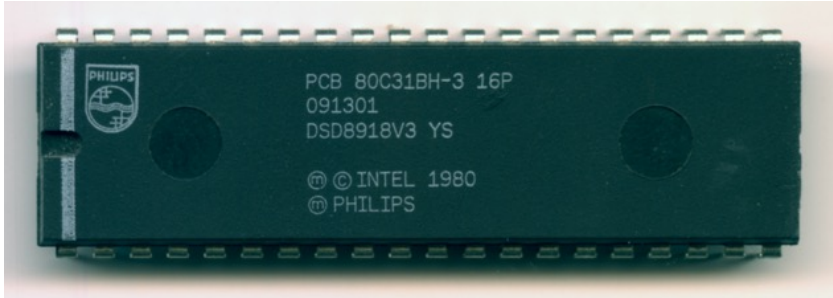
- Modified Harvard architecture.
- Single-chip microcontroller.
- 8-bit, 16-bit address space
- On-chip flash semiconductor memory for your program, and EEPROM and SRAM.
- RISC microprocessor: '…the 32 registers are directly connected to the Arithmetic Logic Unit (ALU), allowing two independent registers to be accessed in a single instruction executed in one clock cycle. The resulting architecture is more code efficient while achieving throughputs up to ten times faster than conventional CISC microcontrollers.'
- There are several sub-families, distinguished by different amounts of memory, package and peripherals.



Arduino UNO programmer in used at Lancaster for a variety of modules

Other relevant families: 8051 family (MCS-51, 80C51)

- Internal Harvard architecture.
- CISC (Complex Instruction Set Computer) processor.
- Developed in 1980 by Intel.
- 8, 16 and 32-bit variants.
- Many different vendors.
- Literally hundreds of different models.



An example of an 8051 microcontroller, Wikipedia

Other relevant families: Motorola MC68HC08 series

- Long-standing.
- CISC microprocessor.
- Von Neumann architecture.

Other relevant families: PIC

- *Peripheral Interface Controller*, made by Microchip Technology.
- Harvard architecture, RISC basis.
- 8, 16 and 32-bit.
- Massive variety in device specifications.
- Small number of fixed-length instructions.
- Entry-level, easy to use, hobbyists.

http://www.microchip.com

Other relevant families: Raspberry Pi

- USB powered
- Operating system from an SD card, ‘Raspbian’
- Educational/developing country focus
- Broadcom SoC / ARM Cortex – early smart phone chip equivalent.
- HDMI / composite video
- Best selling British computer
- https://vimeo.com/90103691